

Relatório Técnico - DR2 Assessment

Renato Hack

1. Escopo

Este relatório documenta a resolução dos exercícios q1 a q12 do assessment de DR2, voltado à programação em Assembly AArch64. A organização do material seguiu as exigências do enunciado até a seção anterior a “Formato da Entrega”.

Cada exercício foi separado em sua própria pasta, com estrutura de arquivos, compilação, geração de objetos, executáveis e validação prática por execução e depuração.

2. Organização dos Arquivos

Cada pasta qX contém os seguintes artefatos:

- `main.S`: código principal, casos de teste e pontos de parada para depuração.
- `lib.S`: implementação da rotina solicitada no exercício.
- `main.o`: arquivo objeto gerado a partir de `main.S`.
- `lib.o`: arquivo objeto gerado a partir de `lib.S`.
- `qX`: executável final da questão.

A estrutura final do trabalho ficou organizada como:

- `q1`, `q2`, ..., `q12`
- relatório técnico em PDF

3. Ambiente Utilizado

Todas as implementações e validações foram preparadas para execução em ambiente WSL, utilizando emulação AArch64 por meio do `qemu-aarch64`.

Ferramentas utilizadas:

- `aarch64-linux-gnu-gcc` para montagem e ligação dos binários AArch64.
- `qemu-aarch64` para execução dos programas.
- `gdb-multiarch` para depuração remota.
- `pandoc` e `pdflatex` para geração deste relatório em PDF.

Os programas foram escritos como binários stand-alone, com `_start` como ponto de entrada e uso direto de syscall `exit` para encerramento.

4. Fluxo de Compilação e Execução

O fluxo padrão adotado nos exercícios foi:

```
aarch64-linux-gnu-gcc -g -c main.S -o main.o
aarch64-linux-gnu-gcc -g -c lib.S -o lib.o
```

```
aarch64-linux-gnu-gcc -g -nostdlib -static -no-pie main.o lib.o -o qX
qemu-aarch64 ./qX
echo $?
```

Para depuração, foi utilizado o modo remoto do QEMU:

```
qemu-aarch64 -g 2401 ./qX
```

Em outro terminal:

```
gdb-multiarch -q ./qX
```

E, dentro do GDB:

```
target remote :2401
```

Esse fluxo substitui a execução nativa em Raspberry Pi, mantendo a mesma lógica de inspeção de registradores, memória e pontos de parada.

5. Critérios Técnicos Atendidos

As soluções foram escritas respeitando a convenção AAPCS64, incluindo:

- passagem de parâmetros inteiros e ponteiros por `x0` a `x7` ou `w0` a `w7`;
- passagem de parâmetros `double` por registradores `d0`, `d1` e seguintes;
- retorno de valores nos registradores esperados;
- alinhamento de pilha em 16 bytes quando necessário;
- preservação de registradores callee-saved quando aplicável.

Além disso, os exercícios atenderam aos requisitos funcionais do enunciado:

- uso explícito de instruções de comparação e desvio;
- implementação direta de laços em Assembly;
- acesso explícito à memória por meio de `ldr`, `str`, `ldrb`, `strb`, `ldrsh` e variantes;
- uso de aritmética inteira, ponto fixo e ponto flutuante conforme o contexto;
- tratamento de erros por códigos de status.

6. Descrição Técnica por Exercício

6.1. q1 - `normalize_saturate_i16_to_i32`

Foi implementada a normalização de amostras PCM `int16_t` para `int32_t`, com extensão de sinal, multiplicação por ganho inteiro e saturação no intervalo `[-223, 223 - 1]`.

A rotina valida:

- ponteiro de entrada nulo;
- ponteiro de saída nulo;
- quantidade de amostras igual a zero.

O `main.S` prepara valores extremos para validar saturação e também casos adicionais para verificação dos códigos de status.

6.2. q2 - `sin_taylor`

A função calcula o seno por série de Taylor em `double`, acumulando termos enquanto:

- `fabs(termo) >= eps`; e
- o número máximo de iterações ainda não foi atingido.

Também foram tratados:

- `eps <= 0`;
- `max_iter == 0`.

O `main.S` valida os casos `x = 0`, `x = 1.0`, epsilon inválido e zero iterações.

6.3. q3 - `dot_i32_checked`

Foi implementado o produto escalar entre dois vetores `int32_t`, com:

- extensão para 64 bits na multiplicação;
- acumulação em `int64_t`;
- detecção de overflow no acumulador por flag de overflow.

O programa principal inclui um caso simples e outro propositalmente preparado para provocar overflow e permitir inspeção em GDB.

6.4. q4 - `safe_div_s64`

A divisão assinada segura trata explicitamente:

- divisor igual a zero, retornando saturação positiva e status `1`;
- caso `INT64_MIN / -1`, retornando saturação positiva e status `2`;
- divisão normal, retornando status `0`.

O `main.S` cobre os três caminhos de execução.

6.5. q5 - `convert_speed_q16`

Foi implementada uma conversão de velocidades em formato `Q16.16`, com seleção por modo:

- `0`: km/h para m/s;
- `1`: m/s para km/h;
- `2`: knots para m/s.

As contas intermediárias utilizam 64 bits e divisão inteira. O modo inválido retorna valor zero e status `1`.

6.6. q6 - `to_lower_ascii_inplace`

A função percorre uma string ASCII até o terminador `0x00`, convertendo apenas caracteres entre `A` e `Z` para minúsculas.

Também realiza:

- contagem de quantas substituições foram feitas;
- retorno de erro para ponteiro nulo.

O caso de teste principal usa a string "HeLLo!", permitindo validar o contador e o resultado final.

6.7. q7 - sqrt_nr

Foi implementada uma aproximação da raiz quadrada em `double` usando Newton-Raphson.

A rotina trata:

- erro para `a < 0`;
- erro para `max_iter == 0`;
- chute inicial `a`, quando `a >= 1`;
- chute inicial `1.0`, quando `a < 1`.

O laço executa exatamente o número de iterações solicitado.

6.8. q8 - trapz_energy_f64

A integração numérica pela regra do trapézio foi implementada em `double`, com validação de:

- ponteiro nulo;
- `N < 2`;
- `dt <= 0`.

O cálculo percorre os pares consecutivos do vetor, acumulando:

```
(f[i] + f[i + 1]) * dt * 0.5
```

6.9. q9 - affine2d_q16

Foi implementada uma transformação afim 2D em `Q16.16` com dois modos:

- `mode 0`: `out = A * p`;
- `mode 1`: `out = A * p + b`.

As multiplicações usam intermediários de 64 bits e normalização posterior por deslocamento de 16 bits. Os ponteiros são validados antes do processamento.

6.10. q10 - moving_avg_i32

A média móvel foi implementada com acumulador deslizando em 64 bits.

Comportamento adotado:

- para `i < W - 1`, a saída recebe zero;
- a partir da janela completa, soma-se o novo elemento;
- quando aplicável, subtrai-se explicitamente `in[i - W]`;
- o resultado é obtido por divisão inteira pela largura da janela.

Foram tratados casos de ponteiro nulo, `N == 0` e janela inválida.

6.11. q11 - poly_horner_q16_sat

A avaliação polinomial em Q16.16 foi implementada pelo método de Horner:

```
acc = coef[0]
```

```
acc = ((acc * x) >> 16) + coef[i]
```

O acumulador é saturado no intervalo $[-2^{31}, 2^{31} - 1]$. Quando a saturação ocorre, a função retorna status 3.

O `main.S` inclui um caso simples e outro propositalmente preparado para saturar.

6.12. q12 - Mini Projeto do Pipeline IMU

O exercício final integra múltiplas rotinas em um pipeline de diagnóstico:

- `normalize_saturate_i16_to_i32`
- `dot_i32_checked`
- `safe_div_s64`
- `sqrtnr`
- `classify_health`

Fluxo implementado:

1. normalização das amostras `int16_t`;
2. cálculo da energia por produto escalar do vetor consigo mesmo;
3. cálculo da média por divisão segura por `N`;
4. conversão explícita do resultado para `double`;
5. cálculo da magnitude por `sqrtnr`;
6. classificação final por faixas.

Foi adotada uma bitmask de flags para propagação de falhas:

- bit 0: ponteiro nulo;
- bit 1: `N` inválido;
- bit 2: erro de divisão ou erro numérico equivalente;
- bit 3: overflow ou saturação relevante ao diagnóstico.

No caso de teste montado, ocorre saturação ainda na normalização. Por isso, a classificação final do pipeline é `FAULT`, e o executável encerra com código 2, o que corresponde exatamente ao comportamento esperado.

7. Validação Realizada

As validações foram conduzidas em dois níveis:

- execução do binário no `qemu-aarch64`, para verificar o código final de saída;
- depuração com `gdb-multiarch`, conectando-se ao stub remoto aberto pelo `qemu-aarch64 -g`.

Resumo dos códigos de saída finais:

- q1 a q11: 0
- q12: 2

Principais checkpoints observados durante a depuração:

- q1: vetor de saída 8388607, -8388608, 617000, -8388608, 0
- q2: $\sin(0) = 0$ e $\sin(1.0) \approx 0.8414709848078937$
- q3: status 3 com acumulador 9223372028264841218
- q4: saturação positiva para divisor zero e para INT64_MIN / -1
- q5: três modos válidos e um modo inválido com status 1
- q6: string final "hello!" e contador 3
- q7: $\sqrt{4.0} = 2.0$, com erro para entrada negativa
- q8: integral 2.0 para [0, 1, 2] com dt = 1
- q9: saída (3, 4) em Q16.16 para $A = I$, $b = (1,1)$ e $p = (2,3)$
- q10: saída [0, 1, 2, 3] com W = 2
- q11: valor $45 \ll 16 = 2949120$ e saturação em 2147483647
- q12: vetor normalizado, energia, magnitude e código final 2 confirmados em GDB

8. Decisões de Engenharia

Algumas decisões foram tomadas para manter o código simples, determinístico e fácil de depurar:

- testes embutidos em main.S, reduzindo dependências externas;
- labels de checkpoint no main.S, facilitando breakpoints no GDB;
- validações internas por comparação com resultados esperados;
- uso de códigos de saída simples para indicar sucesso ou erro;
- no q12, retorno do próprio diagnóstico final como exit code.